

Repaso para el Primer Parcial

Unidad 2 | Ejercicio 4 - Listado

Crear un procedimiento que muestre todos los departamentos y los empleados que trabajan en él.

Recibir el ID de la localidad.

- Si la localidad no existe, informar
- Mostrar el total de empleados
- En caso que nadie trabaje en el departamento mostrar el mensaje correspondiente
- Ordenar alfabéticamente por apellido y nombre.

Mostrar de la siguiente manera:

Localidad: AAAAAAA

Departamento: 1 - XXXXXXXXXXXX

Emp_id	Nombre	Apellido
999999	xxxxxxx	xxxxxxx
999999	xxxxxxx	xxxxxxx
999999	xxxxxxx	xxxxxxx

Total: empleados 3

Departamento: 2 - XXXXXXXXXXXX

Emp_id	Nombre	Apellido
999999	xxxxxxx	xxxxxxx
999999	xxxxxxx	xxxxxxx
999999	xxxxxxx	xxxxxxx

Total: empleados 2

Resolución:

DECLARE

pi_loc_id location.location_id%type;

cursor c_dep is

SELECT department_id, name

FROM department

WHERE location_id = pi_loc_id;

CURSOR c_emp (p_dep_id department.department_id%type) IS

SELECT first_name, last_name

FROM employee

WHERE department_id = p_dep_id;

BEGIN

pi_loc_id := 122;

FOR r_dep IN c_dep LOOP

dbms_output.put_line(r_dep.department_id||' '||r_dep.name);

FOR r_emp IN c_emp(r_dep.department_id) LOOP

dbms_output.put_line(' '||r_emp.first_name||' '||r_emp.last_name);

END LOOP;

dbms_output.put_line('-----');

END LOOP;

END;

COMPLETAR!!!

Unidad 2 | Ejercicio 3 - *Función y procedimiento de listado*

- 1) Realizar una función que recibe un salario y devuelve el valor del id de la escala de salarios
- 2) Hacer procedimiento que recibe una escala de salarios y liste todos los empleados que pertenecen a dicha escala de salarios.
 - Listar para cada empleado: Nombre, apellido, nombre del departamento, nombre de la tarea, salario.
 - Informar la cantidad total de empleados de la escala.
 - Si la escala no existe o no tiene empleados, informarlo por pantalla
 - Es requisito utilizar la función realizada en el punto anterior

Tip. La función también se puede utilizar en una query

Resolución

1)

```
CREATE OR REPLACE
FUNCTION FU_GRADE_ID
(
    PI_SALARIO IN employee.salary%type
) RETURN NUMBER AS
    v_sg_id SALARY_GRADE.grade_id%type;
BEGIN
    SELECT grade_id
    INTO v_sg_id
    FROM salary_grade
    WHERE PI_SALARIO BETWEEN lower_bound
                        AND upper_bound;

    RETURN v_sg_id;
END FU_GRADE_ID;
```

2)

```
CREATE OR REPLACE PROCEDURE PR_LISTE_EMP_SAL
(
    PI_ESCALA IN SALARY_GRADE.GRADE_ID%TYPE DEFAULT 1
) AS
CURSOR c_emp IS
SELECT first_name, last_name, d.name, j.FUNCTION, e.SALARY
FROM employee e,
    DEPARTMENT d,
    job j
WHERE PI_ESCALA = FU_GRADE_ID (e.salary)
    AND e.DEPARTMENT_ID = d.DEPARTMENT_ID
    AND e.JOB_ID = j.JOB_ID;

    v_cant_emp NUMBER;
    v_hay_grado NUMBER;
BEGIN
    v_cant_emp := 0;
    FOR r_emp IN c_emp LOOP
        v_cant_emp := v_cant_emp + 1;
        DBSM_OUTPUT.PUT_LINE(r_emp.first_name||' '||r_emp.last_name||' '||r_emp.name||'
'||r_emp.FUNCTION);
    END LOOP;
```

```

IF v_cant_emp = 0 THEN
    BEGIN
        SELECT 1
        INTO v_hay_grado
        FROM salary_grade
        WHERE grade_id = pi_escala;

        DBMS_OUTPUT.PUT_LINE('No hay empleados');
    EXCEPTION
        WHEN no_data_found THEN
            DBMS_OUTPUT.PUT_LINE('No existe la escala');
    END;
ELSE
    DBMS_OUTPUT.PUT_LINE('No existe la escala');
END IF;
END PR_LISTA_EMP_SAL;

```

Ejercicio de clase

Realizar una función que recibe nombre y apellido del empleado y retorne su ID o cancele con excepciones propias.

Realice un procedimiento que recibe nombre y apellido del empleado y borre todos sus clientes asociados.

- Utilice la función del punto anterior.
- Indique la cantidad de clientes borrados. Si no pudo borrar, indique el problema exacto.
- El procedimiento no debe cancelar.

Resolución

```

CREATE OR REPLACE FUNCTION FU_EMP_ID
(
    PI_NOMBRE IN EMPLOYEE.FIRST_NAME%TYPE,
    PI_APELLIDO IN EMPLOYEE.LAST_NAME%TYPE
) RETURN EMPLOYEE.EMPLOYEE_ID%TYPE AS
    v_emp_id EMPLOYEE.EMPLOYEE_ID%TYPE;
BEGIN
    SELECT employee_id
    INTO v_emp_id
    FROM employee
    WHERE first_name = pi_nombre
        AND last_name = pi_apellido;
    RETURN v_emp_id;
EXCEPTION
    WHEN no_data_found THEN
        raise_application_error(-20001,'No existe empleado');
    WHEN too_many_rows THEN
        raise_application_error(-20002,'Existe mas de un empleado');
    WHEN others THEN
        raise_application_error(-20003,'Error inesperado '|| sqlerrm);
END FU_EMP_ID;

```

```
CREATE OR REPLACE PROCEDURE PR_CORRA_CLI
(
    PI_NOMBRE IN EMPLOYEE.FIRST_NAME%TYPE
    PI_APELLIDO IN EMPLOYEE.LAST_NAME%TYPE
) AS
```

```

    e_fk exception;
    PRAGMA EXCEPTION_init (e_fk, -2292);
    e_NDF exception;
    PRAGMA EXCEPTION_init (e_NDF, -20001);
    e_TMR exception;
    PRAGMA EXCEPTION_init (e_TMR, -20002);
    e_OTH exception;
    PRAGMA EXCEPTION_init (e_OTH, -20003);
```

```
BEGIN
```

```

    DELETE
    FROM customer
    WHERE salesperson_id = FU_EMP_ID(PI_NOMBRE, PI_APELLIDO);
```



Ø
 1
 N
 error!

¡Tenemos
 que tratar
 estas
 posibles
 salidas!

```

    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('El empleado no tiene clientes asociados');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Se borraron '|| SQL%ROWCOUNT ||' clientes');
    END IF;
```

```
EXCEPTION
```

```

    WHEN e_fk THEN
        DBMS_OUTPUT.PUT_LINE('No se puede borrar porque hay cliente/s con ordenes');
    WHEN e_NDF THEN
        DBMS_OUTPUT.PUT_LINE('No existe el empleado');
    WHEN e_TMR THEN
        DBMS_OUTPUT.PUT_LINE('Existe más de un empleado con el mismo nombre');
    WHEN e_OTH THEN
        DBMS_OUTPUT.PUT_LINE('Se profujo un error al buscar el empleado '||sqlerrm);
```

```

    WHEN others THEN
        DBMS_OUTPUT.PUT_LINE('Se produjo un error inesperado '|| sqlerrm);
```

```
END PR_BORRA_CLI;
```